אינטגרציית Cisco DNA Center ל-SolarWinds — הסבר

> איך `Scripts/DNA.DiscoverNodesAndInterfaces.ps1` מושך את מלאי המכשירים מ-Cisco DNA Center, רושם כל מכשיר כ-Node ב-SolarWinds, מוסיף לניטור רק את הממשקים שנבחרו, ומתעד כל פעולה כזאת בלוג.

מסמך מקביל ל-SmartConsole-Integration-Explained.pdf, עבור מסלול ה-DNA.

1. מה האוטומציה הזאת עושה

המטרה היא לקחת את כתובות ה-IP הניהוליות של כל המכשירים הרשומים ב-Cisco DNA Center (שנקרא היום Catalyst Center), לרשום כל אחד מהם כ-Node מנוטר ב-SolarWinds, ואז לגלות את הממשקים של אותו Node ולהוסיף לניטור רק את אלה שעומדים בקריטריון מוגדר.

בניגוד למסלול ה-SmartConsole, שמפוצל לארבעה קבצים, כאן הכול נמצא בסקריפט אחד שרץ מקצה לקצה. הוא עובר את השלבים הבאים לפי הסדר:

שלב	פונקציה / קטע	תפקיד
1	`Write-Log` / `Invoke-LogRotation`	פותח את הלוג של ההרצה ומבצע רוטציה אם הקובץ חרג מהגודל שהוגדר.
2	`Get-TokenDNA`	מתחבר ל-DNA ב-Basic Auth ומחזיר token לשימוש בכל הקריאות הבאות.
3	`Get-DNADevices`	מושך את מלאי המכשירים, עמוד אחרי עמוד, ומחזיר את רשימת המכשירים.
4	`Connect-Swis` + בניית `$compon ents`	מתחבר ל-SolarWinds ומתרגם את הכתובות למבנה אחיד עם הגדרות SNMP.
5	`Add-Component`	יוצר Node חדש ב-SolarWinds ורושם עבורו את ארבעת ה-pollers הסטנדרטיים.
6	`Add-DiscoveredInterfaces`	מגלה את ממשקי ה-Node, מסנן לפי רשימת היתר, ומוסיף את הנותרים לניטור.
7	`Stop-Sessions`	סוגר את החיבור ל-SWIS וזורק את ה-token של DNA מהזיכרון.

למה סקריפט אחד, ולא ארבעה כמו במסלול SmartConsole

מסלול ה-SmartConsole מפוצל לפונקציות שנטענות ב-dot-sourcing, עם קובץ CSV אופציונלי בין הגילוי לבין ההוספה — מה שמאפשר שלב סקירה ידני לפני שמשהו נכנס לניטור. הסקריפט הזה בנוי כהרצה אחת רציפה, בלי שלב סקירה: מה שמגיע מ-DNA נכנס לניטור באותה הרצה. זה נוח למשימה מתוזמנת, אבל אומר שטעות בצד ה-DNA מתורגמת מיד ל-Nodes חדשים ולצריכת רישיון. הלוג (פרק 8) הוא מה שמפצה על היעדר שלב הסקירה.

2. התחברות ל-DNA וקבלת token

הפונקציה `Get-TokenDNA` מבצעת `POST` אל `/dna/system/api/v1/auth/token` עם אימות Basic: שם המשתמש והסיסמה משורשרים בפורמט `user:pass`, מקודדים ב-Base64 ונשלחים בכותרת `Authorization`.

```
$pair = "${user}:${pass}"
$encode = [System.Convert]::ToBase64String([text.encoding]::ASCII.GetBytes($pair))
$header = @{
    "Content-Type"  = "application/json"
    "Authorization" = "Basic $encode "
}
$response = Invoke-WebRequest -Uri $uri -Headers $header -Method Post -SkipCertificateCheck
return $response.Content | ConvertFrom-Json
```

הקריאה עטופה ב-`try/catch`, ומיד אחריה נבדק שה-token שחזר אינו ריק. שתי הבדיקות נחוצות בנפרד: שרת שמחזיר `200` עם גוף לא צפוי לא יזרוק שגיאה, ובלי הבדיקה המפורשת הסקריפט היה ממשיך עם token ריק ונכשל רק בשלב הבא, עם הודעה מטעה.

```
try {
    $tokenObj    = Get-TokenDNA -uri $dnaUrlToken -user $dnaCredentials.UserName ...
    $tokenString = $tokenObj.Token

    if (-not $tokenString) {
        Write-Log "DNA authentication did not return a token. Check URL/credentials." -Level ERROR
        Stop-Sessions
        exit
    }
} catch {
    Write-Log "Failed to authenticate with DNA: $($_.Exception.Message)" -Level ERROR
    Stop-Sessions
    exit
}
```

3. משיכת מלאי המכשירים

`Get-DNADevices` שולחת את ה-token בכותרת `x-auth-token` — הכותרת שכל קריאות ה-Intent API של DNA דורשות — ומושכת את רשימת המכשירים.

ה-API מחזיר עמוד אחד בכל קריאה, ולא את כל המלאי. לכן הפונקציה רצה בלולאה עם `offset` ו-`limit` וממשיכה כל עוד העמוד שחזר מלא. כשעמוד חוזר קצר מגודל העמוד — זהו העמוד האחרון.

```
$devices  = @()
$pageSize = 500
$offset   = 1   # the DNA device inventory is 1-based

do {
    $pageUri  = "$($url)?offset=$($offset)&limit=$($pageSize)"
    $response = Invoke-WebRequest -Uri $pageUri -Headers $header -Method Get -SkipCertificateCheck
    $page     = @(($response.Content | ConvertFrom-Json).response)

    $devices += $page
    $offset  += $pageSize
} while ($page.Count -eq $pageSize)

return $devices
```

למה זה קריטי דווקא כאן

קריאה בודדת בלי pagination לא מחזירה שגיאה — היא פשוט מחזירה את העמוד הראשון בלבד. בסביבה עם יותר מכשירים מגודל העמוד, חלק מהמכשירים לא היו נרשמים לניטור ושום הודעה לא הייתה מופיעה. זו הסיבה שהלולאה נבנתה לפי אורך העמוד ולא לפי שדה `total`.

4. התחברות ל-SWIS ובניית רשימת הרכיבים

אחרי שהמלאי בידיים, הסקריפט מוציא ממנו את הכתובות הניהוליות ומוודא שיש מה לעבד לפני שהוא בכלל נוגע ב-SolarWinds:

```
$addresses = @($results.managementIpAddress)

if (-not $addresses) {
    Write-Log "DNA returned no devices - nothing to process." -Level WARN
    Stop-Sessions
    exit
}
```

רק אז נפתח החיבור ל-SWIS, ולאחריו כל כתובת מתורגמת ל-hashtable אחיד שמחזיק, לצד הכתובת, את הגדרות ה-SNMP המשותפות שהגיעו מבלוק ה-`param`:

```
$components = $addresses |
    Where-Object { $_ } |
    ForEach-Object { $_.Split(',') } |
    ForEach-Object { $_.Trim() } |
    Where-Object { $_ } |
    ForEach-Object {
        @{
            IPAddress   = $_
            EngineID    = $EngineID
            SNMPVersion = $SNMPVersion
            DNS         = ""
            SysName     = ""
            Community   = $SNMPCommunity
        }
    }
```

הפיצול ב-`Split(',')` נשאר מהמסלול הקודם: הוא מאפשר גם ערך יחיד וגם מחרוזת עם כמה כתובות מופרדות בפסיק, כך שאותו בלוק היה עובד גם על רשימה שהוזנה ידנית.

5. הוספת Node ורישום pollers

`Add-Component` יוצרת את ה-Node בשתי קריאות SWIS. הראשונה יוצרת את האובייקט ומחזירה URI בלבד; כדי לקבל את ה-`NodeID` — שנדרש לרישום ה-pollers — צריך לשלוף את האובייקט בחזרה בקריאה שנייה.

```
$newNodeUri = New-SwisObject -SwisConnection $swis -EntityType "Orion.Nodes" -Properties
    $newNodeProps
$nodeProps  = Get-SwisObject -SwisConnection $swis -Uri $newNodeUri

foreach ($pollerType in @(
    "N.Status.ICMP.Native",
    "N.ResponseTime.ICMP.Native",
    "N.Details.SNMP.Generic",
    "N.Uptime.SNMP.Generic"
)) {
    $poller["PollerType"] = $pollerType
    New-SwisObject -SwisConnection $swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
    Write-Log "  poller added    | NodeID=$($nodeProps['NodeID']) | pollerType=$pollerType" –
    NoConsole
    $script:stats.PollersAdded++
}

return $nodeProps["NodeID"]
```

ארבעת ה-pollers הם המינימום שהופך Node חדש למנוטר בפועל: זמינות והיענות ב-ICMP, ופרטי מכשיר וזמן פעילות ב-SNMP. בלי רישום מפורש שלהם ה-Node ייווצר, אבל יישאר ריק בממשק. כל `pollerType` שנרשם מתועד בלוג בשורה נפרדת.

6. גילוי הממשקים והסינון

`Add-DiscoveredInterfaces` קוראת ל-verb של NPM שמבצע גילוי ממשקים על ה-Node, ורק אם הגילוי הצליח ממשיכה לסינון.

הסינון הוא **רשימת היתר**: נשמרים רק ממשקי TenGigabit ו-Port-channel שנמצאים במצב תפעולי תקין (`ifOperStatus` שווה 1). כל השאר מוסר לפני ההוספה.

```
@($discovered.DiscoveredInterfaces.DiscoveredLiteInterface) | Where-Object {
    $_.Caption.InnerText -notmatch '^(TenGigabitEthernet|TenGigE|Te\d|Port-channel|Po\d)' -or
    $_.ifOperStatus -ne '1'
} | ForEach-Object { $discovered.DiscoveredInterfaces.RemoveChild($_) | Out-Null }
```

שלוש נקודות שקל לטעות בהן בבלוק הזה

- **התנאי הפוך בכוונה.** הבלוק מסיר את מה ש-`Where-Object` מחזיר, ולכן "השאר רק A ו-B שהם תקינים" נכתב כ-`-notmatch` על השם **או** `-ne '1'` על הסטטוס. שימוש ב-`-and` במקום `-or` היה משאיר בטעות כל ממשק תקין שאינו ברשימת ההיתר.
- **השוואה מדויקת ולא regex.** `-ne '1'` ולא `-match '1'`, כי `-match` על מספר מחפש תת-מחרוזת ולכן היה מתאים גם ל-12 ול-21.
- **מימוש הרשימה ב-`@()` לפני הלולאה.** `RemoveChild` מקצר את ה-`XmlNodeList` החי בזמן שה-pipeline עדיין עובר עליו, וללא המימוש המוקדם חלק מהממשקים היו מדולגים.

לאחר הסינון נספרים הממשקים שנשארו. ה-`Where-Object` בשורת הספירה אינו מיותר: `@($null)` מחזיר `Count` של 1, ולכן בלי הסינון הזה הרצה שסיננה את כל הממשקים הייתה מדווחת על ממשק אחד ועדיין קוראת ל-verb ההוספה בלי כלום.

ההוספה עצמה עטופה ב**השוואת לפני ואחרי**: ממשקי ה-Node נקראים מ-SolarWinds לפני הקריאה ואחריה, וההפרש הוא מה שבאמת נוסף. זה המנגנון שמשותף גם ל-`FMC.DiscoverNodes.ps1` ולסקריפטים של Radware, SmartConsole ו-Nexus Dashboard.

```
$interfaceQuery = "SELECT InterfaceID, Caption FROM Orion.NPM.Interfaces WHERE NodeID = @nodeId"
try {
    $before = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery `
        -Parameters @{ nodeId = $nodeId } | ForEach-Object { $_.InterfaceID })

    Invoke-SwisVerb $swis Orion.NPM.Interfaces AddInterfacesOnNode `
        @($nodeId, $discovered.DiscoveredInterfaces, "AddDefaultPollers") | Out-Null

    $added = @(Get-SwisData -SwisConnection $swis -Query $interfaceQuery `
        -Parameters @{ nodeId = $nodeId } |
        Where-Object { $before -notcontains $_.InterfaceID })

    if ($added.Count -eq 0) {
        Write-Log
    " No new interfaces for node $($nodeId) - the $interfaceCount matching interface[s] were already
    monitored."
        return
    }

    Write-Log " Added $($added.Count) interface[s] for node $($nodeId)."
    foreach ($interface in $added) {
        Write-Log "  interface added | NodeID=$nodeId | InterfaceID=$($interface.InterfaceID) |
    caption=$($interface.Caption)" -NoConsole
    }
    $script:stats.InterfacesAdded += $added.Count
} catch {
    Write-Log " Failed to add interfaces for node $($nodeId): $($_.Exception.Message)" -Level ERROR
}
```

7. הלולאה הראשית וסגירת הסשנים

הלולאה עוברת על הרכיבים, בודקת מול SWIS אם ה-Node כבר קיים, ומוסיפה רק אם לא. בשני המקרים — חדש או קיים — היא ממשיכה לגילוי הממשקים, כך שגם Node ותיק מקבל את הממשקים שהתווספו אליו מאז. כל אחד משלושת המצבים נרשם בלוג.

```
foreach ($component in $components) {
    Write-Log "Processing component $($component.IPAddress)..."

    try {
        $existing = Get-SwisData -SwisConnection $swis `
            -Query "SELECT NodeID FROM Orion.Nodes WHERE IPAddress = '$($component.IPAddress)'"

        if ($existing) {
            $nodeId = $existing
            Write-Log " Node already exists {NodeID $nodeId}, skipping add."
            Write-Log "  component       | IP=$($component.IPAddress) | status=exists |
   NodeID=$nodeId" -NoConsole
            $script:stats.ComponentsExisting++
        }
        else {
            $nodeId = Add-Component $component
            Write-Log " Added node [NodeID $nodeId]."
            Write-Log "  component       | IP=$($component.IPAddress) | status=added |
   NodeID=$nodeId" -NoConsole
            $script:stats.ComponentsAdded++
        }

        Add-DiscoveredInterfaces $nodeId
    } catch {
        Write-Log " Failed to process $($component.IPAddress): $($_.Exception.Message)" -Level ERROR
        Write-Log "  component       | IP=$($component.IPAddress) | status=failed" -NoConsole
        $script:stats.ComponentsFailed++
    }
}
```

`Stop-Sessions` נקראת גם בסיום התקין וגם לפני כל אחת מנקודות היציאה המוקדמות, כדי ששגיאה בשלב ה-DNA לא תדלג על הניקוי.

> **שני הצדדים אינם סימטריים**
>
> **DNA:** ל-Catalyst Center אין endpoint לביטול token. הטוקן תקף שעה ופג מעצמו, ולכן בניגוד ל-logout האמיתי שמבצע סקריפט ה-SmartConsole, כאן אפשר רק לזרוק אותו מהזיכרון.
>
> **SolarWinds:** אין cmdlet בשם `Disconnect-Swis`, אבל `Connect-Swis` מחזיר אובייקט `InfoServiceProxy` שמממש `IDisposable` — ולכן הסגירה בצד הזה אמיתית.

8. הלוג

כל הרצה מתעדת את עצמה לקובץ לוג. זה מה שמפצה על היעדר שלב סקירה: אחרי הרצה מתוזמנת אפשר לפתוח את הלוג ולראות בדיוק אילו Nodes נוספו, אילו כבר היו קיימים, אילו ממשקים נכנסו לניטור ואילו pollers נרשמו.

מודל הלוג: קובץ אחד עם רוטציה לפי גודל

הלוג בנוי באותו מודל של `DNS.SetSolarWindsRecordToActiveServer.ps1`: קובץ יחיד ב-`-LogPath`, שעובר רוטציה כשהוא מגיע ל-`-LogMaxBytes`, עם `-LogKeep` דורות קודמים לצידו.

פרמטר	ברירת מחדל	מה הוא קובע
`-LogPath`	—	קובץ הלוג, כברירת מחדל `%ProgramData%\SolarWinds\dna-discover-nodes.log`. התיקייה נוצרת בכתיבה הראשונה אם אינה קיימת.
`-LogMaxBytes`	`1MB`	הגודל שבו מתבצעת רוטציה. טווח מותר: 4KB עד 100MB.
`-LogKeep`	`3`	כמה דורות קודמים לשמור. ערך 0 מוחק את הלוג ומתחיל מחדש.

הרוטציה מזיזה את הדורות אחד אחורה — `.log` הופך ל-`.log.1`, `.log.1` ל-`.log.2` וכן הלאה — והישן ביותר נופל:

```
if (-not (Test-Path -LiteralPath $Path)) { return }
if ((Get-Item -LiteralPath $Path).Length -lt $script:LogMaxBytes) { return }

# Nothing to keep: the log simply starts over.
if ($script:LogKeep -lt 1) {
    Remove-Item -LiteralPath $Path -Force -ErrorAction Stop
    return
}

# Oldest generation falls off the end.
$oldest = "$Path.$($script:LogKeep)"
if (Test-Path -LiteralPath $oldest) {
    Remove-Item -LiteralPath $oldest -Force -ErrorAction Stop
}

# Shift the rest down, highest first so nothing is overwritten on the way.
for ($i = $script:LogKeep - 1; $i -ge 1; $i--) {
    $from = "$Path.$i"
    if (Test-Path -LiteralPath $from) {
        Move-Item -LiteralPath $from -Destination "$Path.$($i + 1)" -Force -ErrorAction Stop
    }
}

Move-Item -LiteralPath $Path -Destination "$Path.1" -Force -ErrorAction Stop
```

> **למה רוטציה לפי גודל ולא מחיקה לפי גיל**
>
> המודל הזה חוסם את הנפח הכולל ב-`(LogKeep + 1) × LogMaxBytes` — כ-4MB בברירת המחדל — ללא תלות בתדירות ההרצה. מחיקה לפי גיל, לעומת זאת, משאירה את הנפח תלוי בכמה הרצות היו בחלון הזמן: משימה שרצה כל חמש דקות תייצר פי שנים-עשר יותר קבצים ממשימה שרצה כל שעה, באותה מדיניות שמירה בדיוק.

מה נרשם

כל שורה נושאת חותמת זמן ורמה (`INFO`, `WARN` או `ERROR`) בפורמט קבוע. שורות הפירוט — אחת לכל רכיב, ממשק ו-poller — נכתבות לקובץ בלבד, כדי שהקונסולה תישאר קריאה גם בהרצה על מאות מכשירים:

```
2026-10-05 13:32:21 [INFO ] ===== Run started =====
2026-10-05 13:32:21 [INFO ] Processing component 10.20.30.41...
2026-10-05 13:32:21 [INFO ]   component       | IP=10.20.30.41 | status=added | NodeID=118
2026-10-05 13:32:21 [INFO ]   poller added    | NodeID=118 | pollerType=N.Status.ICMP.Native
2026-10-05 13:32:21 [INFO ]   poller added    | NodeID=118 | pollerType=N.ResponseTime.ICMP.Native
2026-10-05 13:32:21 [INFO ]  Added 2 interface[s] for node 118.
2026-10-05 13:32:21 [INFO ]   interface added | NodeID=118 | InterfaceID=901 |
    caption=TenGigabitEthernet1/0/1
2026-10-05 13:32:21 [INFO ]   interface added | NodeID=118 | InterfaceID=902 | caption=Port-channel1
2026-10-05 13:32:21 [INFO ] Processing component 10.20.30.42...
2026-10-05 13:32:21 [INFO ]  Node already exists {NodeID 77}, skipping add.
2026-10-05 13:32:21 [INFO ]   component       | IP=10.20.30.42 | status=exists | NodeID=77
2026-10-05 13:32:21 [INFO ]  No new interfaces for node 77 - the 3 matching interface[s] were already
    monitored.
2026-10-05 13:32:21 [INFO ] Summary: 1 component[s] added, 1 already existed, 0 failed; 2
    interface[s] and 2 poller[s] added.
```

סוג רשומה	מה היא מכילה
`component`	כתובת ה-IP, הסטטוס (`added` , `exists` או `failed`) וה-`NodeID` .
`poller added`	ה-`NodeID` וה-`pollerType` שנרשם — שורה לכל אחד מארבעת ה-pollers.
`interface added`	ה-`NodeID` , ה-`InterfaceID` וה-caption. נרשמים רק ממשקים שההוספה באמת יצרה; ממשק שכבר היה מנוטר אינו מקבל שורה.
Summary	שורת סיכום אחת בסוף: כמה רכיבים נוספו, כמה כבר היו קיימים, כמה נכשלו, וכמה ממשקים ו-pollers נרשמו בסך הכול.

מכיוון שהגילוי מחזיר כל ממשק תואם על המכשיר — מנוטר או לא — בלי השוואת הלפני/אחרי הרצה חוזרת על Node שלא השתנה הייתה רושמת שוב את אותם ממשקים כ"נוספו", והסכום בשורת הסיכום היה מנופח. Node שכל הממשקים התואמים שלו כבר מנוטרים מקבל עכשיו שורה אחת (`No new interfaces...`) במקום רשימה. ה-caption בשורת הלוג מגיע מ-SolarWinds ולא מה-XML של הגילוי, ולכן זה השם שהממשק נושא בפועל במערכת.

כשל בכתיבה אינו מפיל את ההרצה

כתיבת הלוג עטופה ב- `try/catch` שמסתפק באזהרה. הרציונל פשוט: אובדן שורת לוג אינו סיבה לנטוש גילוי שכבר רץ.

```powershell
$line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
if (-not $NoConsole) {
    Write-Information -MessageData $line -InformationAction Continue
}

try {
    $directory = Split-Path -Path $script:LogPath -Parent
    if ($directory -and -not (Test-Path -LiteralPath $directory)) {
        New-Item -Path $directory -ItemType Directory -Force -ErrorAction Stop | Out-Null
    }
    Invoke-LogRotation -Path $script:LogPath
    Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -ErrorAction Stop
}
catch {
    Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
}
```

סטייה מכוונת מסקריפט ה-DNS

סקריפט ה-DNS מגדיר `$ErrorActionPreference = 'Stop'` ברמת הסקריפט כולו, וזה מה שהופך את פקודות הקובץ החשופות שלו ל-catchable. הסקריפט הזה אינו מגדיר זאת, ולכן פקודות הקובץ כאן נושאות `-ErrorAction Stop` משלהן. בלי התוספת הזאת כשל כתיבה היה חומק מה-`catch` ומודפס כשגיאה גולמית במקום כאזהרה המתוכננת.

9. זרימה מקצה לקצה

שלב	מה קורה	אם זה נכשל
1	פתיחת הלוג ורוטציה אם צריך	אזהרה; ההרצה ממשיכה בלי לוג
2	קבלת token מ-DNA	שגיאה, `Stop-Sessions`, יציאה
3	משיכת המלאי, עמוד אחר עמוד	שגיאה, `Stop-Sessions`, יציאה
4	חילוץ הכתובות הניהוליות	אזהרה אם אין מכשירים, ויציאה
5	התחברות ל-SWIS	שגיאה, `Stop-Sessions`, יציאה
6	לכל כתובת: הוספת Node אם אינו קיים	נרשם ללוג כ-`failed`, ממשיך לכתובת הבאה
7	גילוי, סינון והוספת ממשקים	נרשם ללוג, ממשיך לכתובת הבאה
8	שורת סיכום וסגירת הסשנים	אזהרה בלבד

ההבדל בין שלבים 2–5 לשלבים 6–7 מכוון: כשל בהתחברות עוצר הכול, כי בלעדיו אין מה לעשות; כשל על מכשיר בודד רק נרשם ללוג, כדי שמכשיר אחד בעייתי לא יעצור את כל ההרצה.

10. מה לשנות לפני הרצה אמיתית

- **למלא את פרטי החיבור.** `$hostname`, `$username`, `$dnaServer` ונתיב ה-clixml של אישורי ה-DNA (`$dnaCredentials`) מגיעים ריקים ומחכים לעריכה. הנתיב הריק של אישורי ה-DNA יזרוק שגיאה מיד בהרצה.
- **להגדיר community אמיתי.** `$SNMPCommunity` ריק כברירת מחדל; Node שיווצר עם community ריק לא יצליח לבצע polling ב-SNMP.
- **לאמת את ה-EngineID.** ברירת המחדל היא 2; בסביבה עם polling engine יחיד הערך הנכון הוא בדרך כלל 1.
- **לוודא הרשאת כתיבה ל-`-LogPath`.** ברירת המחדל יושבת תחת `%ProgramData%`; חשבון המשימה המתוזמנת צריך להיות מורשה לכתוב שם, אחרת ההרצה תעבוד אך לא תתועד.
- **לאמת את ה-endpoint ואת ה-pagination.** הסקריפט פונה ל-`network-devices`, בעוד שה-endpoint המתועד ברוב הגרסאות הוא `network-device`. כדאי לוודא מול הגרסה שרצה אצלכם שגם השם וגם הפרמטרים `offset` / `limit` נתמכים.
- **PowerShell 7 ומעלה.** `-SkipCertificateCheck` אינו קיים ב-Windows PowerShell 5.1, ושם הסקריפט ייפול.
- **להריץ קודם על קבוצה קטנה.** אין שלב סקירה ואין הרצה יבשה — כל מכשיר שמגיע מ-DNA נרשם לניטור וצורך רישיון.

11. מגבלות ידועות

דברים שעדיין פתוחים בקוד, ונרשמים כאן כדי שלא יתגלו בהפתעה:

- **אין סינון לפי זמינות.** כל מכשיר שמופיע ב-DNA נרשם, גם אם ה-`reachabilityStatus` שלו אינו תקין — מה שעלול לייצר גל התראות ביום הראשון.
- **המטא-דאטה של DNA לא מנוצלת.** מתוך כל מה ש-DNA מחזיר נלקחת רק `managementIpAddress`, והשדות `DNS` ו-`SysName` נשארים ריקים, למרות ש-DNA מספק גם `hostname`, `platformId`, `family` ו-`serialNumber`.
- **אין סינון כפילויות.** אם DNA מחזיר את אותה כתובת פעמיים, היא תעובד פעמיים.
- **שאילתת ה-SWQL בנויה בשרשור מחרוזות** ולא כשאילתה פרמטרית.
- **תת-ממשקים נכללים.** תבנית הסינון בודקת את תחילת השם בלבד, ולכן `TenGigabitEthernet1/0/1.100` ייכלל גם הוא.
- **הפילטר תלוי בצורת ה-XML.** `$_.Caption.InnerText` מחזיר ערך רק כאשר אלמנט `Caption` נושא attribute, כפי שתגובת SWIS מוטיפסת אכן מחזירה. אילו השרת היה מחזיר אלמנט טקסט נקי, מתאם ה-XML של PowerShell היה משטח אותו למחרוזת, `.InnerText` היה ריק, והפילטר היה מסיר כל ממשק בשקט. זה נוגע כיום לפילטר בלבד — שורות הלוג כבר לוקחות את ה-caption מ-SolarWinds.
- **קוד היציאה אינו משתנה בכשל.** גם יציאה מוקדמת אחרי שגיאה מסיימת בקוד 0, כך שמתזמן משימות לא יזהה את הכשל אלא מתוך הלוג.
- **כפילות מול סקריפט ה-SmartConsole.** `Add-Component`, הלולאה הראשית ובניית רשימת הרכיבים כמעט זהים בשני הקבצים, ותיקון באחד אינו מגיע לשני.

מסמך זה מתאר את `Scripts/DNA.DiscoverNodesAndInterfaces.ps1` שבמאגר. קטעי הקוד מצוטטים מהסקריפט כפי שהוא, ומקוצרים במקומות שבהם הושמטו הערות או שורות שאינן נחוצות להסבר. פונקציות הלוג נבדקו מול PowerShell 7.4.6; הסקריפט עצמו לא נבדק מול הרצה חיה מול DNA ו-SolarWinds.